МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №8

з дисципліни **"Функціональне програмування"**

на тему **"АЛГЕБРАЇЧНІ ТИПИ ДАНИХ "ПО-PYTHON'ІВСЬКИ""**

Варіант 14. Події системи

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: описати журнал подій системи як алгебраїчний тип даних засобами Python — незмінними `dataclass`-вузлами, `Union`/`TypeAlias`, `Literal` і `Protocol`; обробити журнал чистою функцією-reducer та реалізувати статистику й текстове представлення через `match/case`, перевіривши типи `mypy --strict`.

Теоретичні відомості:

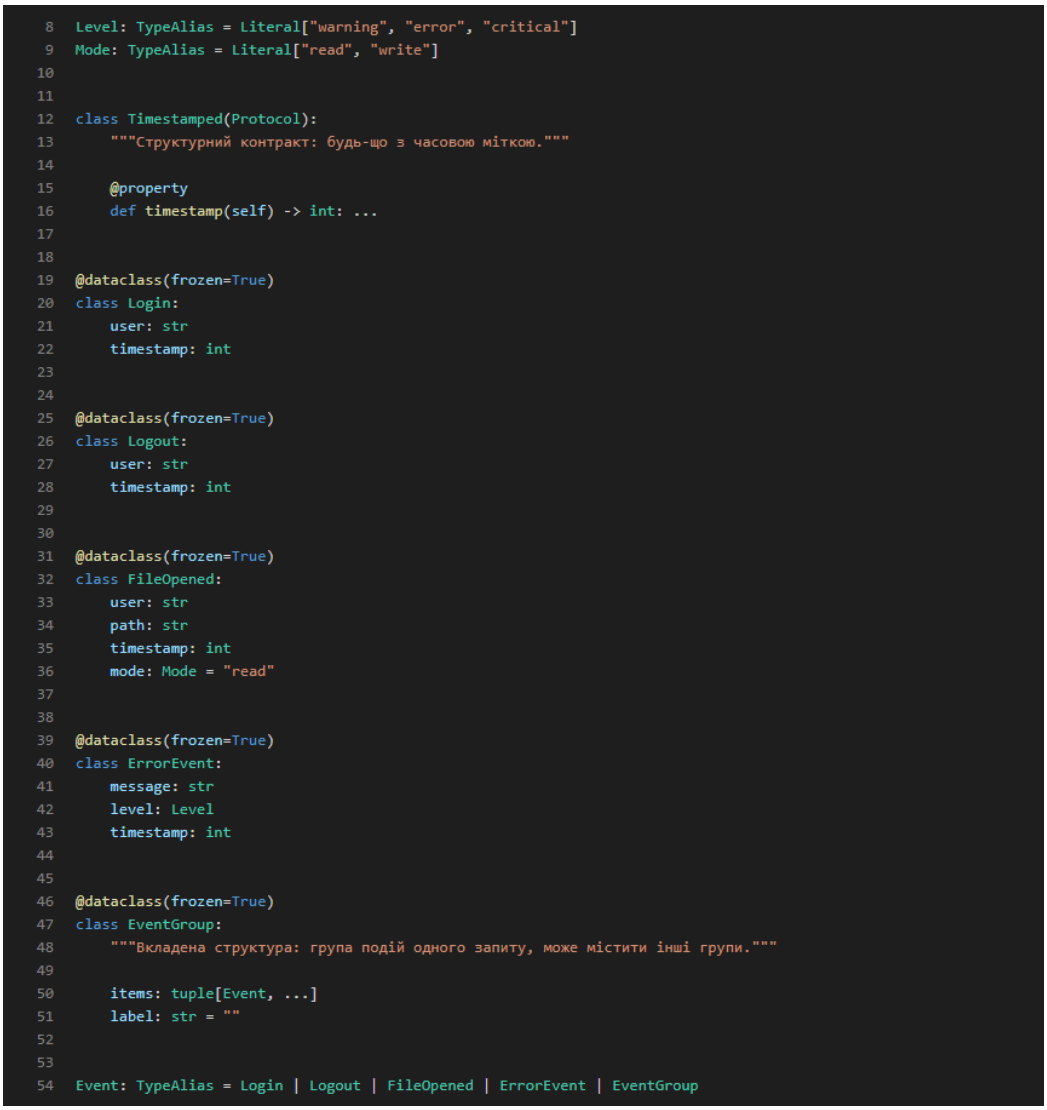
Алгебраїчний тип-сума — це значення, яке належить рівно одному з кількох варіантів, кожен зі своїм набором полів. У Python варіанти описують як `@dataclass(frozen=True)`, а об'єднання — як `Union` з іменем через `TypeAlias`. Заморожені вузли хешовані й незмінні, тому будь-яка функція над ними змушена повертати нове значення. `Literal` обмежує поле скінченним набором рядків (рівень помилки, режим доступу до файлу), і mypy відхилить невідомий рівень ще на етапі перевірки типів. `Protocol` задає структурний контракт: подія «з часовою міткою» — це будь-який об'єкт, що має атрибут `timestamp`, без спільного базового класу.

Структурний pattern matching `match/case` розбирає значення за формою: шаблон `case ErrorEvent(level="critical", message=m)` одночасно перевіряє тип, звіряє літерал і прив'язує поле до змінної, замінюючи ланцюжки `isinstance` та `if/elif`. Обробка журналу зводиться до чистого reducer-а `step(state, event) → state`: згортка `functools.reduce` пропускає початковий стан через усі події, а кожен крок будує новий словник статистики, не змінюючи попередній. Для рекурсивної структури (група подій, що містить інші групи) той самий reducer викликається рекурсивно, тому вкладеність обробляється природно.

Хід роботи:

1. У `model.py` описала ADT подій за варіантом 14: `Literal`-псевдоніми `Level` (рівень помилки) та `Mode` (режим відкриття файлу), `Protocol Timestamped` із

властивістю `timestamp`, незмінні вузли `Login`, `Logout`, `FileOpened`, `ErrorEvent` і рекурсивний `EventGroup` з кортежем вкладених подій; тип `Event` — їх `Union`:



```python
Level: TypeAlias = Literal["warning", "error", "critical"]
Mode: TypeAlias = Literal["read", "write"]


class Timestamped(Protocol):
    """Структурний контракт: будь-що з часовою міткою."""

    @property
    def timestamp(self) -> int: ...


@dataclass(frozen=True)
class Login:
    user: str
    timestamp: int


@dataclass(frozen=True)
class Logout:
    user: str
    timestamp: int


@dataclass(frozen=True)
class FileOpened:
    user: str
    path: str
    timestamp: int
    mode: Mode = "read"


@dataclass(frozen=True)
class ErrorEvent:
    message: str
    level: Level
    timestamp: int


@dataclass(frozen=True)
class EventGroup:
    """Вкладена структура: група подій одного запиту, може містити інші групи."""

    items: tuple[Event, ...]
    label: str = ""


Event: TypeAlias = Login | Logout | FileOpened | ErrorEvent | EventGroup
```

2. В `evaluator.py` визначила `TypedDict Stats` і незмінний початковий стан `EMPTY`, перевірку часової мітки (від'ємна → `ValueError`), числову вагу рівня

`level_rank` через `match` по літералах:

```python
class Stats(TypedDict):
    logins: int
    logouts: int
    files_opened: int
    errors: dict[Level, int]
    active_users: frozenset[str]
    last_timestamp: int


EMPTY: Stats = {
    "logins": 0,
    "logouts": 0,
    "files_opened": 0,
    "errors": {},
    "active_users": frozenset(),
    "last_timestamp": 0,
}
"""Початковий стан reducer-а; ніколи не мутується — step повертає новий словник."""


def _check_ts(ts: int) -> int:
    if ts < 0:
        raise ValueError(f"negative timestamp: {ts}")
    return ts


def level_rank(level: Level) -> int:
    """Числова вага рівня помилки для порівнянь."""
    match level:
        case "warning":
            return 1
        case "error":
            return 2
        case "critical":
            return 3

```

3. Додала допоміжні чисті функції: `kind` повертає назву виду події, `flatten` рекурсивно розгортає групу у кортеж листових подій, `timestamp_of` дістає мітку через or-шаблони, а для групи — найпізнішу серед вкладених:

```python
def kind(e: Event) -> str:
    """Назва виду події для статистики та виводу."""
    match e:
        case Login():
            return "login"
        case Logout():
            return "logout"
        case FileOpened():
            return "file"
        case ErrorEvent():
            return "error"
        case EventGroup():
            return "group"


def flatten(e: Event) -> tuple[Event, ...]:
    """Усі листові події групи у порядку обходу (новий кортеж)."""
    match e:
        case EventGroup(items=items):
            return tuple(leaf for i in items for leaf in flatten(i))
        case _:
            return (e,)


def timestamp_of(e: Event) -> int:
    """Часова мітка події; для групи — найпізніша серед вкладених (0 для порожньої)."""
    match e:
        case EventGroup(items=items):
            return max((timestamp_of(i) for i in items), default=0)
        case Login(timestamp=t) | Logout(timestamp=t):
            return _check_ts(t)
        case FileOpened(timestamp=t) | ErrorEvent(timestamp=t):
            return _check_ts(t)

```

4. Центральна частина — reducer `step`: кожна гілка `match` повертає новий словник стану (кількість входів, виходів, відкритих файлів, помилки за рівнями, множина активних користувачів, остання мітка), а для `EventGroup` стан згортається рекурсивно через `reduce`; `reduce_log` обробляє весь журнал одним викликом `reduce`:



```python
def step(state: Stats, e: Event) -> Stats:
    """Крок reducer-а: повертає НОВИЙ стан, вхідний не змінюється."""
    last = max(state["last_timestamp"], timestamp_of(e))
    match e:
        case Login(user=u):
            return {
                **state,
                "logins": state["logins"] + 1,
                "active_users": state["active_users"] | {u},
                "last_timestamp": last,
            }
        case Logout(user=u):
            return {
                **state,
                "logouts": state["logouts"] + 1,
                "active_users": state["active_users"] - {u},
                "last_timestamp": last,
            }
        case FileOpened():
            return {
                **state,
                "files_opened": state["files_opened"] + 1,
                "last_timestamp": last,
            }
        case ErrorEvent(level=lvl):
            errors = {**state["errors"], lvl: state["errors"].get(lvl, 0) + 1}
            return {**state, "errors": errors, "last_timestamp": last}
        case EventGroup(items=items):
            return reduce(step, items, state)


def reduce_log(events: Iterable[Event], initial: Stats = EMPTY) -> Stats:
    """Згортає журнал подій у статистику однією чистою функцією reduce."""
    return reduce(step, events, initial)
```

5. Функції статистики помилок: `error_level` через `match` повертає рівень лише для `ErrorEvent`, `count_errors` рахує помилки не нижче заданого рівня,

`worst_level` знаходить найвищий рівень у журналі з урахуванням вкладених груп:

```python
def error_level(e: Event) -> Level | None:
    """Рівень помилки для ErrorEvent, None для решти подій."""
    match e:
        case ErrorEvent(level=lvl):
            return lvl
        case _:
            return None


def _error_levels(events: Iterable[Event]) -> list[Level]:
    leaves = (leaf for e in events for leaf in flatten(e))
    return [lvl for lvl in map(error_level, leaves) if lvl is not None]


def count_errors(events: Iterable[Event], *, min_level: Level = "warning") -> int:
    """Кількість помилок рівня не нижче min_level (з урахуванням вкладених груп)."""
    threshold = level_rank(min_level)
    return sum(1 for lvl in _error_levels(events) if level_rank(lvl) >= threshold)


def worst_level(events: Iterable[Event]) -> Level | None:
    """Найвищий рівень помилки в журналі або None, якщо помилок немає."""
    levels = _error_levels(events)
    if not levels:
        return None
    worst: Level = max(levels, key=level_rank)
    return worst
```

6. У `pretty.py` описала `Protocol` `Renders` і рекурсивний `describe`, який рендерить подію у текст; запис у файл і критична помилка мають власні шаблони з літералами `mode="write"` та `level="critical"`, групи виводяться з відступом за рівнем вкладеності:

```python
class Renders(Protocol):
    """Структурний контракт для будь-чого, що вміє відрендерити себе."""

    def render(self) -> str: ...


def describe(e: Event, indent: int = 0) -> str:
    """Багаторядковий опис події; групи рендеряться рекурсивно з відступом."""
    pad = "  " * indent
    match e:
        case Login(user=u, timestamp=t):
            return f"{pad}[{t}] login {u}"
        case Logout(user=u, timestamp=t):
            return f"{pad}[{t}] logout {u}"
        case FileOpened(user=u, path=p, timestamp=t, mode="write"):
            return f"{pad}[{t}] {u} writes {p}"
        case FileOpened(user=u, path=p, timestamp=t):
            return f"{pad}[{t}] {u} opens {p}"
        case ErrorEvent(message=m, level="critical", timestamp=t):
            return f"{pad}[{t}] !!! CRITICAL: {m}"
        case ErrorEvent(message=m, level=lvl, timestamp=t):
            return f"{pad}[{t}] {lvl}: {m}"
        case EventGroup(items=items, label=label):
            head = f"{pad}group {label or '(unnamed)'} [{len(items)}]"
            return "\n".join([head, *(describe(i, indent + 1) for i in items)])
```

7. Базове завдання з методички: AST `Expr = Num | Add | Mul` і альтернативна

форма з одним бінарним вузлом `Bin`, де оператор — `Literal["+", "*"]`:

```python
@dataclass(frozen=True)
class Num:
    value: int


@dataclass(frozen=True)
class Add:
    left: Expr
    right: Expr


@dataclass(frozen=True)
class Mul:
    left: Expr
    right: Expr


Expr: TypeAlias = Num | Add | Mul

Op: TypeAlias = Literal["+", "*"]


@dataclass(frozen=True)
class Num2:
    value: int


@dataclass(frozen=True)
class Bin:
    op: Op
    left: Expr2
    right: Expr2


Expr2: TypeAlias = Num2 | Bin
```

```python
class SupportsBin(Protocol):
    """Структурний контракт бінарного вузла."""

    @property
    def op(self) -> Op: ...

    @property
    def left(self) -> Expr2: ...

    @property
    def right(self) -> Expr2: ...


def eval_expr2(e: Expr2) -> int:
    """Чистий обчислювач альтернативного AST з оператором-літералом."""
    match e:
        case Num2(value=v):
            return v
        case Bin(op="+", left=a, right=b):
            return eval_expr2(a) + eval_expr2(b)
        case Bin(op="*", left=a, right=b):
            return eval_expr2(a) * eval_expr2(b)
        case _:
            raise ValueError(f"unknown node: {e!r}")
```

8. Чистий обчислювач `eval_expr` у `eval_.py` з перевіркою діапазону

(`EvalOverflow`) та `render` для дужкового запису:

```python
MAX_ABS: Final[int] = 10_000


class EvalOverflow(ValueError):
    """Результат вийшов за межі ±MAX_ABS."""


def _check_range(x: int) -> int:
    if abs(x) > MAX_ABS:
        raise EvalOverflow(f"overflow: {x}")
    return x


def eval_expr(e: Expr) -> int:
    """Обчислює вираз рекурсивно; кожен проміжний результат перевіряється."""
    match e:
        case Num(value=v):
            return _check_range(v)
        case Add(left=a, right=b):
            return _check_range(eval_expr(a) + eval_expr(b))
        case Mul(left=a, right=b):
            return _check_range(eval_expr(a) * eval_expr(b))


def render(e: Expr) -> str:
    """Повністю дужковий текст виразу."""
    match e:
        case Num(value=v):
            return str(v)
        case Add(left=a, right=b):
            return f"({render(a)} + {render(b)})"
        case Mul(left=a, right=b):
            return f"({render(a)} * {render(b)})"
```

9. Демонстраційний `main.py` будує журнал із вкладеною групою запиту (відкриття файлів і група повторних спроб із попередженням та помилкою), критичною помилкою та виходом користувача, друкує опис, статистику reducer-а, кількість помилок від рівня `error`, найгірший рівень і обчислення виразів:

```python
def main() -> None:
    """Друкує опис журналу, статистику reducer-а та підсумок помилок."""
    log = (
        Login("alice", 100),
        EventGroup(
            label="request-7",
            items=(
                FileOpened("alice", "/etc/config.yml", 105),
                FileOpened("alice", "/var/log/app.log", 106, mode="write"),
                EventGroup(
                    label="retry",
                    items=(
                        ErrorEvent("disk quota exceeded", "warning", 107),
                        ErrorEvent("write failed", "error", 108),
                    ),
                ),
            ),
        ),
        Login("bob", 110),
        ErrorEvent("database unreachable", "critical", 115),
        Logout("alice", 120),
    )
    for e in log:
        print(describe(e))
    print("stats:")
    for key, value in reduce_log(log).items():
        print(f"  {key}: {value}")
    print("errors >= error:", count_errors(log, min_level="error"))
    print("worst level:", worst_level(log))

    ast = Mul(Add(Num(2), Num(3)), Num(4))
    print("(2 + 3) * 4 =", eval_expr(ast))
    print("Bin:", eval_expr2(Bin("+", Num2(2), Bin("*", Num2(3), Num2(4)))))
```

10. Написала pytest-тести для подій: кожен вид події у `step` повертає новий стан, повна статистика журналу та незмінність вхідного кортежу, рекурсивна група згортається так само, як послідовність її листків:

```python
LOGIN = Login("alice", 100)
OPEN = FileOpened("alice", "/etc/config.yml", 105)
WRITE = FileOpened("alice", "/var/log/app.log", 106, mode="write")
WARN = ErrorEvent("disk quota exceeded", "warning", 107)
ERR = ErrorEvent("write failed", "error", 108)
CRIT = ErrorEvent("database unreachable", "critical", 115)
LOGOUT = Logout("alice", 120)
GROUP = EventGroup(
    label="request-7",
    items=(OPEN, WRITE, EventGroup(label="retry", items=(WARN, ERR))),
)
LOG: tuple[Event, ...] = (LOGIN, GROUP, Login("bob", 110), CRIT, LOGOUT)


def test_step_each_kind_returns_new_state() -> None:
    s1 = step(EMPTY, LOGIN)
    assert s1["logins"] == 1 and s1["active_users"] == frozenset({"alice"})
    assert s1["last_timestamp"] == 100
    s2 = step(s1, OPEN)
    assert s2["files_opened"] == 1 and s2["last_timestamp"] == 105
    s3 = step(s2, ERR)
    assert s3["errors"] == {"error": 1}
    s4 = step(s3, LOGOUT)
    assert s4["logouts"] == 1 and s4["active_users"] == frozenset()
    assert EMPTY == {
        "logins": 0,
        "logouts": 0,
        "files_opened": 0,
        "errors": {},
        "active_users": frozenset(),
        "last_timestamp": 0,
    }


def test_reduce_log_full_statistics() -> None:
    before = deepcopy(LOG)
    stats = reduce_log(LOG)
    assert stats == {
        "logins": 2,
        "logouts": 1,
        "files_opened": 2,
        "errors": {"warning": 1, "error": 1, "critical": 1},
        "active_users": frozenset({"bob"}),
        "last_timestamp": 120,
    }
    assert LOG == before
    assert reduce_log(()) == EMPTY
    assert reduce_log(()) is EMPTY


def test_group_is_reduced_recursively() -> None:
    assert step(EMPTY, GROUP) == reduce_log((OPEN, WRITE, WARN, ERR))
    assert step(EMPTY, EventGroup(items=())) == EMPTY

```

11. Тести `flatten`/`kind`/`timestamp_of`, рівнів і підрахунку помилок, від'ємної часової мітки, незмінності та хешованості вузлів, рендеру вкладеної групи та

структурного контракту `Timestamped`:

```python
def test_flatten_kind_and_timestamp() -> None:
    assert flatten(GROUP) == (OPEN, WRITE, WARN, ERR)
    assert flatten(CRIT) == (CRIT,)
    assert [kind(e) for e in LOG] == ["login", "group", "login", "error", "logout"]
    assert kind(OPEN) == "file"
    assert timestamp_of(GROUP) == 108 and timestamp_of(EventGroup(items=())) == 0
    assert timestamp_of(LOGOUT) == 120


def test_levels_and_error_counts() -> None:
    assert [level_rank(lvl) for lvl in ("warning", "error", "critical")] == [1, 2, 3]
    assert count_errors(LOG) == 3
    assert count_errors(LOG, min_level="error") == 2
    assert count_errors(LOG, min_level="critical") == 1
    assert count_errors((LOGIN, LOGOUT)) == 0
    assert worst_level(LOG) == "critical"
    assert worst_level((GROUP,)) == "error"
    assert worst_level((LOGIN,)) is None
    assert error_level(ERR) == "error" and error_level(LOGIN) is None


def test_negative_timestamp_rejected() -> None:
    with pytest.raises(ValueError):
        timestamp_of(Login("x", -1))
    with pytest.raises(ValueError):
        reduce_log((EventGroup(items=(OPEN, ErrorEvent("boom", "error", -5))),))


def test_nodes_are_immutable_and_hashable() -> None:
    with pytest.raises(FrozenInstanceError):
        LOGIN.user = "mallory"  # type: ignore[misc]
    assert LOGIN == Login("alice", 100)
    assert len({LOGIN, Login("alice", 100), LOGOUT}) == 2
    assert hash(GROUP) == hash(deepcopy(GROUP))


def test_describe_nested() -> None:
    assert describe(GROUP).splitlines() == [
        "group request-7 [3]",
        "  [105] alice opens /etc/config.yml",
        "  [106] alice writes /var/log/app.log",
        "  group retry [2]",
        "    [107] warning: disk quota exceeded",
        "    [108] error: write failed",
    ]
    assert describe(LOGIN) == "[100] login alice"
    assert describe(LOGOUT) == "[120] logout alice"
    assert describe(CRIT) == "[115] !!! CRITICAL: database unreachable"
    assert describe(EventGroup(items=())) == "group (unnamed) [0]"


def test_timestamped_protocol() -> None:
    def when(e: Timestamped) -> int:
        return e.timestamp

    assert [when(e) for e in (LOGIN, OPEN, ERR, LOGOUT)] == [100, 105, 108, 120]
    e: Event = CRIT
    assert isinstance(e, ErrorEvent) and e.level == "critical"
```

12. Тести базового завдання з методички: обчислення, вкладений вираз, рендер,

переповнення та форма `Bin`:

```python
def test_eval_basic() -> None:
    assert eval_expr(Num(5)) == 5
    assert eval_expr(Add(Num(2), Num(3))) == 5
    assert eval_expr(Mul(Num(2), Num(3))) == 6


def test_eval_nested() -> None:
    ast = Mul(Add(Num(2), Num(3)), Num(4))
    assert eval_expr(ast) == 20


def test_render() -> None:
    ast = Mul(Add(Num(2), Num(3)), Num(4))
    assert render(ast) == "((2 + 3) * 4)"


def test_overflow() -> None:
    with pytest.raises(EvalOverflow):
        eval_expr(Mul(Num(1000), Num(1000)))


def test_bin_literal_form() -> None:
    assert eval_expr2(Bin("+", Num2(2), Bin("*", Num2(3), Num2(4)))) == 14
```

13. Property-based тести на Hypothesis з рекурсивними стратегіями: довільний журнал із вкладеними групами дає ту саму статистику, що й його листки, лічильники узгоджені з кількістю подій і `count_errors`, `step` не мутує переданий

стан, а рендер випадкового виразу обчислюється до того ж значення, що й `eval_expr`:

```python
ts = st.integers(min_value=0, max_value=10_000)
user = st.sampled_from(["alice", "bob", "carol"])
leaf = st.one_of(
    st.builds(Login, user, ts),
    st.builds(Logout, user, ts),
    st.builds(FileOpened, user, st.just("/f"), ts, st.sampled_from(["read", "write"])),
    st.builds(
        ErrorEvent,
        st.just("msg"),
        st.sampled_from(["warning", "error", "critical"]),
        ts,
    ),
)
event = st.recursive(
    leaf,
    lambda inner: st.builds(EventGroup, st.lists(inner, max_size=4).map(tuple)),
    max_leaves=12,
)
log = st.lists(event, max_size=6).map(tuple)


@given(log)
def test_group_reduces_like_flat_leaves(events: tuple[Event, ...]) -> None:
    leaves = tuple(leaf for e in events for leaf in flatten(e))
    assert reduce_log(events) == reduce_log(leaves)
    assert reduce_log((EventGroup(items=events),)) == reduce_log(events)


@given(log)
def test_counts_are_consistent(events: tuple[Event, ...]) -> None:
    s = reduce_log(events)
    leaves = tuple(leaf for e in events for leaf in flatten(e))
    assert s["logins"] + s["logouts"] + s["files_opened"] + sum(
        s["errors"].values()
    ) == len(leaves)
    assert sum(s["errors"].values()) == count_errors(events)
    assert count_errors(events, min_level="critical") <= count_errors(events)
    assert s["last_timestamp"] == max(map(timestamp_of, leaves), default=0)


@given(event)
def test_step_does_not_mutate_state(e: Event) -> None:
    state = step(EMPTY, e)
    before = deepcopy(state)
    step(state, e)
    assert state == before and EMPTY["errors"] == {}
```

14. Запустила демонстрацію: журнал із вкладеною групою рендериться з відступами, reducer порахував два входи, один вихід, два відкриті файли та по одній помилці кожного рівня; активним лишився лише bob:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab08> python main.py
[100] login alice
group request-7 [3]
  [105] alice opens /etc/config.yml
  [106] alice writes /var/log/app.log
  group retry [2]
    [107] warning: disk quota exceeded
    [108] error: write failed
[110] login bob
[115] !!! CRITICAL: database unreachable
[120] logout alice
stats:
  logins: 2
  logouts: 1
  files_opened: 2
  errors: {'warning': 1, 'error': 1, 'critical': 1}
  active_users: frozenset({'bob'})
  last_timestamp: 120
errors >= error: 2
worst level: critical
(2 + 3) * 4 = 20
Bin: 14
```

15. Перевірила проєкт автоматичними інструментами: усі 18 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab08> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 18 items

tests\test_events.py::test_step_each_kind_returns_new_state PASSED       [  5%]
tests\test_events.py::test_reduce_log_full_statistics PASSED             [ 11%]
tests\test_events.py::test_group_is_reduced_recursively PASSED           [ 16%]
tests\test_events.py::test_flatten_kind_and_timestamp PASSED             [ 22%]
tests\test_events.py::test_levels_and_error_counts PASSED                [ 27%]
tests\test_events.py::test_negative_timestamp_rejected PASSED            [ 33%]
tests\test_events.py::test_nodes_are_immutable_and_hashable PASSED       [ 38%]
tests\test_events.py::test_describe_nested PASSED                        [ 44%]
tests\test_events.py::test_timestamped_protocol PASSED                   [ 50%]
tests\test_expr.py::test_eval_basic PASSED                               [ 55%]
tests\test_expr.py::test_eval_nested PASSED                              [ 61%]
tests\test_expr.py::test_render PASSED                                   [ 66%]
tests\test_expr.py::test_overflow PASSED                                 [ 72%]
tests\test_expr.py::test_bin_literal_form PASSED                         [ 77%]
tests\test_properties.py::test_group_reduces_like_flat_leaves PASSED     [ 83%]
tests\test_properties.py::test_counts_are_consistent PASSED              [ 88%]
tests\test_properties.py::test_step_does_not_mutate_state PASSED         [ 94%]
tests\test_properties.py::test_render_is_evaluable_python PASSED         [100%]

============================= 18 passed in 1.13s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab08> python -m mypy --strict .
Success: no issues found in 10 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab08> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab08> python -m black --check .
All done! ✨ 🍰 ✨
10 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я змоделювала події системи (варіант 14) як алгебраїчний тип даних: незмінні `dataclass`-вузли `Login`, `Logout`, `FileOpened`, `ErrorEvent`, `Union`-псевдонім `Event`, `Literal` для рівнів помилок і режимів доступу та `Protocol Timestamped` для подій із часовою міткою. Журнал обробляється чистим reducer-ом `step`/`reduce_log`, який повертає новий стан на кожному кроці, а статистика, підрахунок помилок і текстовий опис реалізовані через `match/case` без ланцюжків `isinstance`; рекурсивний вузол `EventGroup` показав, що pattern matching і згортка природно обходять вкладені структури. Юніт- і property-тести з рекурсивними стратегіями Hypothesis, `mypy --strict`, ruff і black підтвердили коректність і типобезпечність моделі.